UNIVERSIDADE FEDERAL FLUMINENSE

ENGENHARIA DE TELECOMUNICAÇÕES

SISTEMAS DE COMPUTAÇÃO PARA TELECOMUNICAÇÕES

TET 00197



SISTEMA OPERACIONAL ANDROID

Grupo:

RAFAEL CAVEARI GOMES

JEAN ALVES R. FERNANDES

VINICIUS CORRÊA FERREIRA

Professora:

NATALIA CASTRO FERNANDES

10 de julho de 2012

Sumário

Lista de Figuras **3**

1 Introdução **4**

1.1 Gerenciamento de processos 4

1.2 Gerenciamento de memória 4

1.3 Sistema de arquivos 5

1.4 Entrada e saída de dados 5

2 Objetivo e motivação **5**

3 Estrutura do Sistema *Android* **7**

3.1 O *kernel* 9

3.2 O sistema de arquivos 11

3.3 Os serviços básicos 12

3.4 As aplicações 13

3.5 O processo de boot 14

3.5.1 O init 16

3.5.2 O Zygote 17

3.5.3 System server 17

4 Gerenciamento de processos e memória **17**

4.1 *Android* runtime 18

4.2 Dalvik Virtual Machine 18

4.2.1 Prioridade e *status* de processos 19

4.3 Memória Virtual 20

4.3.1 Segmentação 21

4.3.2 Paginação 21

4.3.3 Substituição de páginas 22

4.4 Comunicação entre processos 22

4.5 Escalonamento de *CPU* 23

4.6 *Deadlocks* 24

5 Desenvolvimento no *Android* **24**

5.1 Android.os 26

6 Considerações finais **26**

A Tabela de funções da *API* android.os **27**

Referências **29**

Lista de Figuras

1 Evolução da quantidade de celulares no mundo. *Fonte: UIT, Wireless Intelligence, GSA/Informa e Teleco.* 6

2 Arquitetura do sistema operacional Android. *Fonte: http://developer.android.com/.* . 8

3 Detalhes da arquitetura interna do sistema *Android. Fonte: http://developer.android.com/.* 10

4 Exemplo de log no *Android. Fonte: [11].* 11

5 Exemplo de uma árvore de diretórios de um sistema de arquivos no *Android. Fonte: [11].* 12

6 Executáveis que fazem um link para o *Toolbox. Fonte: [11].* 13

7 Exemplo de processos rodando no *Android. Fonte: [11].* 14

8 Esquema da *beagleboard. Fonte: [11].* 15

9 Estados de processos e seu nível de prioridade associado. *Fonte: [4].* 19

10 Paginação no *Linux*. 22

11 Fluxograma de solicitação de página. 23

12 Funções de interfaces da *API* android.os. *Fonte: http://developer.android.com/.* . . . 27

13 Funções de exceptions da *API* android.os. *Fonte: http://developer.android.com/.* . . . 27

14 Funções de enums da *API* android.os. *Fonte: http://developer.android.com/.* 27

15 Funções de classes da *API* android.os. *Fonte: http://developer.android.com/.* 28

1 Introdução

Um computador é composto basicamente por uma Unidade Central de Processamento (CPU), memória e dispositivos de entrada e saída. O responsável pelo controle e alocação de recursos para este sistema é o programa chamado sistema operacional. Sistemas operacionais são programas com a função de gerir o *hardware* de um computador. O sistema operacional age como uma interface entre o usuário e o *hardware*, provendo uma base para a execução de programas. As principais funções de um sistema operacional são portanto:

- Gerenciamento de processos.
- Gerenciamento de memória.
- Sistema de arquivos.
- Entrada e saída de dados.

1.1 Gerenciamento de processos

A maioria dos computadores modernos são sistemas multitarefa, ou seja, são feitos para dar ao usuário a sensação de que múltiplos processos e programas estão sendo executados simultaneamente. É função do sistema operacional coordenar a ordenação e execução de cada processo. Além disto existe também a comunicação entre os processos, conhecido como *Inter–Process Communication (IPC)*, que também deve ser fornecida pelo sistema operacional.

1.2 Gerenciamento de memória

O sistema operacional possui acesso à memória e coordena a utilização desta por processos dos usuários e garante a utilização segura da mesma. Grande parte dos sistemas operacionais utilizam o conceito de memória virtual. O sistema deve portanto assegurar que cada processo tenha seu próprio espaço na memória, prover a proteção deste espaço para que não haja a sobrescrição e utilização por outro processo e possibilitar que uma aplicação não utilize mais memória que a existente fisicamente.

1.3 Sistema de arquivos

A memória principal do computador é volátil, isto é, todo o seu conteúdo é perdido quando a alimentação é desligada, e seu tamanho é limitado pelo custo do hardware. Assim, os usuários necessitam de algum método para armazenar e recuperar informações de modo permanente. Para uma utilização futura os dados devem ser armazenados em um dispositivo periférico não voláteis, como um disco rígido *(HD)*, *CD*, etc, que pode ser lido e gravado por um ou mais processos.

1.4 Entrada e saída de dados

É necessário haver a entrada de dados no sistema, para que estes sejam processados e gerem as informações desejadas pelo usuário. Em computadores pessoais geralmente são utilizados como dispositivos de entrada o mouse, o teclado ou a leitura dos dados é feita diretamente de dispositivo de memória secundária, como *CDs (Compact Discs)* e *HDs (Hard Disks)*. E para o usuário ter acesso a estas informações geradas necessita-se também de um dispositivo de saída de dados. Sendo os mais comumente utilizados a tela, as caixas de som ou é feita a escrita em dispositivos de memória secundário. Todas estas ações são realizadas pelo sistema operacional.

2 Objetivo e motivação

Estudos mostram que nos dias atuais mais de seis bilhões de pessoas possuem um celular, como visto na Figura 1, e isto é quase a totalidade da população mundial que é de sete bilhões de habitantes, segundo a *ONU*.

"A crescente evolução dos dispositivos móveis, impulsionada pelo modo em que os fabricantes vêm trazendo aparelhos cada vez mais completos, tem aberto um novo mercado de aplicações para celular. A presença de *GPS* (Sistema de Posicionamento Global), conexão *Wi–Fi*, acelerômetros (sensores de movimento) e aumento na capacidade de processamento, fazem com que estes aparelhos estejam aptos a portar aplicações mais robustas e prover novas funcionalidades. Aparelhos mais robustos requerem sistemas operacionais mais complexos, e com capacidade de gerenciar os novos componentes de seu hardware. Sendo assim, alguns sistemas operacionais surgiram para sanar este ponto, dentre eles pode–se citar: *Symbian OS*, *Iphone OS*, *Google Android* e *Windows Mobile* [10]". Neste trabalho vamos nos aprofundar no sistema operacional *Android*.



Figura 1: Evolução da quantidade de celulares no mundo. *Fonte: UIT, Wireless Intelligence, GSA/Informa e Teleco.*

"*Android* é a plataforma para dispositivos móveis mais popular do mundo. O *Android* está presente em milhões de celulares, *tablets*, e outros dispostivos trazendo o poder da *Google* e da *web* em suas mãos.

Com um browser incrívelmente rápido, sincronização em nuvem, sistema multitarefa, facilidades para se conectar e compartilhar, e os mais recentes aplicativos da *Google* (e milhares de outros aplicativos disponíveis na *Google Play*) com *Android* seu dispositivo está muito além de inteligente" [2].

A história deste sistema inicia–se em julho de 2005 quando a *Google* adquiriu a *Android Inc.*, uma pequena empresa em *Palo Alto*, *California*, *USA* [3]. Na *Google*, no tempo em que era conduzida por Andy Rubin, foi desenvolvida uma plataforma para sistemas móveis baseado em *Linux*, com o objetivo de ser uma plataforma flexível, aberta e de fácil migração para os fabricantes. Nascia assim o embrião do sistema operacional *Android*. Esse sistema foi desenvolvido na plataforma *Java* e atualmente é mantido pela *OHA* (*Open Handset Alliance*), um grupo constituído por aproximadamente 80 empresas (entre as quais estão a *HTC*, *LG*, *Motorola*, *Samsung*, *Sony Ericsson*, *Toshiba*, *Sprint Nextel*, *China Mobile*, *T-Mobile*, *Asus*, *Intel*, *Garmin* e outras mais) as quais se uniram para inovar e acelerar o desenvolvimento de aplicativos e serviços, com a finalidade de trazer aos consumidores uma experiência muito mais rica em termos de recursos e menos custosa em termos financeiros para o mercado de telefonia móvel.

3 Estrutura do Sistema *Android*

Como dito anteriormente, o *Android* é um sistema operacional baseado no *kernel* do *Linux*. Apesar de ter sido desenvolvido inicialmente para *smartphones*, hoje é usado em diversas outras aplicações como *tablets*, *netbooks*, relógios, etc.

Apesar de ser baseado no *kernel* do *Linux*, existe pouca coisa em comum com distribuições *Linux* convencionais (embarcadas ou não), lembrando que um sistema embarcado (ou sistema embutido) é um sistema microprocessado no qual o computador é completamente encapsulado ou dedicado ao dispositivo ou sistema que ele controla [13]. À grosso modo, o *Android* é uma máquina virtual *Java* rodando sobre o *kernel* do *Linux*, dando suporte para o desenvolvimento de aplicações *Java* através de um conjunto de bibliotecas e serviços.

"Muitas características presentes no *Android*, tais como gráficos 3D e suporte a banco de dados nativo, também estão disponíveis em outras plataformas móveis. Porém, apenas no *Android* há um componente que permite exibir e manipular um mapa do *Google Maps*, serviço de mapas do *Google*, dentro de uma aplicação". Somente no *Android* todos os aplicativos são criados igualmente. Ou seja, nele não há distinção entre aplicativos que são nativos e os demais. Isso possibilita uma grande customização do sistema operacional, permitindo a substituição completa de aplicativos nativos por outros, criados por terceiros. Além disto, todos os aplicativos têm acesso as mesmas funcionalidades [12].

Sua arquitetura possui basicamente 4 camadas, como visto na Figura 2. Uma breve descrição das mesmas é mostrada abaixo:

- **Aplicações:** A camada de aplicativos é a que está no topo da pirâmide da arquitetura do sistema operacional *Android*, composta pelo conjunto de aplicações nativas do mesmo. Dentre estes pode–se citar: cliente de *e-mail*, despertador, calendário, jogos, mapas, *browser* e internet, etc.

- **Framework:** A camada de *framework* nativo disponibiliza aos desenvolvedores as mesmas *Applications Programming Interface (APIs)* – Interface de Programação de Aplicativos utilizadas para a criação de aplicações originais do sistema operacional *Android*. Este *framework* permite que o programador tenha o mesmo acesso ao sistema que os aplicativos da camada de aplicativos possuem. Este *framework* foi criado para abstrair a complexidade e simplificar o reutilização de procedimentos. Essa camada funciona como um *link* com a camada de

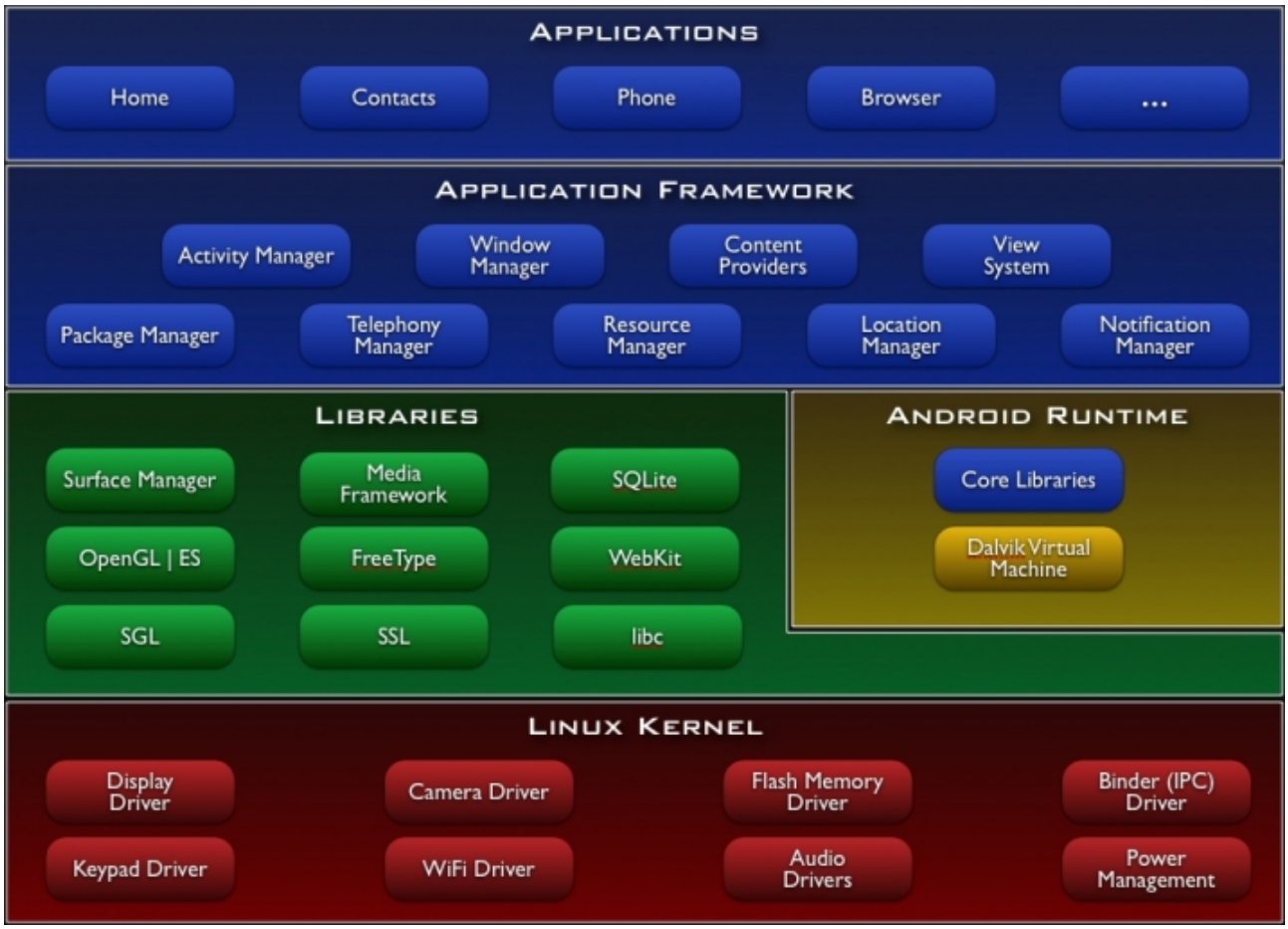


Figura 2: Arquitetura do sistema operacional Android. *Fonte: http://developer.android.com/.*

bibliotecas do sistema operacional que serão acessadas através de *APIs* contidas no *framework*.

- **Bibliotecas e serviços:** Essas bibliotecas são responsáveis por fornecer funcionalidades para manipular o áudio, vídeo, gráficos, banco de dados e *browser*. Algumas bibliotecas são a *Bionic*, a *OpenGL/ES* para trabalhar com interface gráfica, e a *SQLite* para trabalhar com banco de dados. Aqui também estão os serviços usados em camadas superiores, como máquina virtual *Java Dalvik*. (trataremos mais desse assunto na secção 4.2). A maior parte destas bibliotecas e serviços estão desenvolvidos em C e C++;

- **O *Android Runtime*:** Permite que cada *thread* rode sua própria instância da MV (máquina virtual). Embora no desenvolvimento de aplicativos seja utilizada a linguagem *Java*, as aplicações não são executadas em uma máquina virtual *Java* tradicional, e sim em outra chama de *Dalvik*. Essa máquina virtual é otimizada especialmente para dispositivos móveis. A plataforma *Google Android* permite o desenvolvimento de aplicativos na linguagem *Java*. Essa

máquina virtual foi construída pelos engenheiros da *Google*, para obter um consumo mínimo de memória e isolamento de processos. Ela permite que as aplicações escritas em linguagem *Java* sejam executadas normalmente;

- ***Kernel Linux*:** A camada do *kernel* é baseada em um sistema do sistema operacional *Linux* versão 2.6. Esta camada atua também como responsável pela abstração entre o *hardware* e os aplicativos e é responsável pelos serviços principais do sistema operacional Android, como o gerenciamento de memória e de processos. Várias funções do *kernel* são utilizadas diretamente pelo *Android*, mas muitas modificações foram feitas para otimizar memória e tempo de processamento das aplicações. Essas modificações incluem novos dispositivos de *drivers*, adições no sistema de gerenciamento de energia e um sistema que possibilita terminar processos de maneira criteriosa quando há pouca memória disponível. O *Linux* 2.6 foi escolhido por já conter uma grande quantidade de *drivers* de dispositivos sólidos e por ter um bom gerenciamento de memória e processos.

Se olharmos para a arquitetura interna do *Android*, veremos o nível de complexidade deste sistema operacional, vista na Figura 3.

3.1 O *kernel*

Como vimos anteriormente o *Android* usa uma versão modificada do *kernel* do *Linux*. Dentre as principais modificações, podemos citar:

- **Binder:** Em todo sistema operacional com suporte à memória virtual os processos rodam em diferentes regiões de memória. Isso significa que nenhum processo tem acesso direto à região de memória de outro processo ou *thread*. Dessa forma precisamos de um mecanismo de comunicação entre processos. No *Android* é utilizado o *Binder*, que será visto com mais detalhes na secção 4.5.

- **Ashmem:** É um novo mecanismo de compartilhamento de memória, onde dois ou mais processos podem comunicarem-se através de uma região compartilhada de memória. É mais leve e fácil de usar, tem um melhor suporte a dispositivos com pouca memória, já que tem a capacidade de descartar regiões de memória compartilhada de maneira segura em caso de pouca memória disponível. Sua implementação encontra-se em “mm/ashmem.c”.

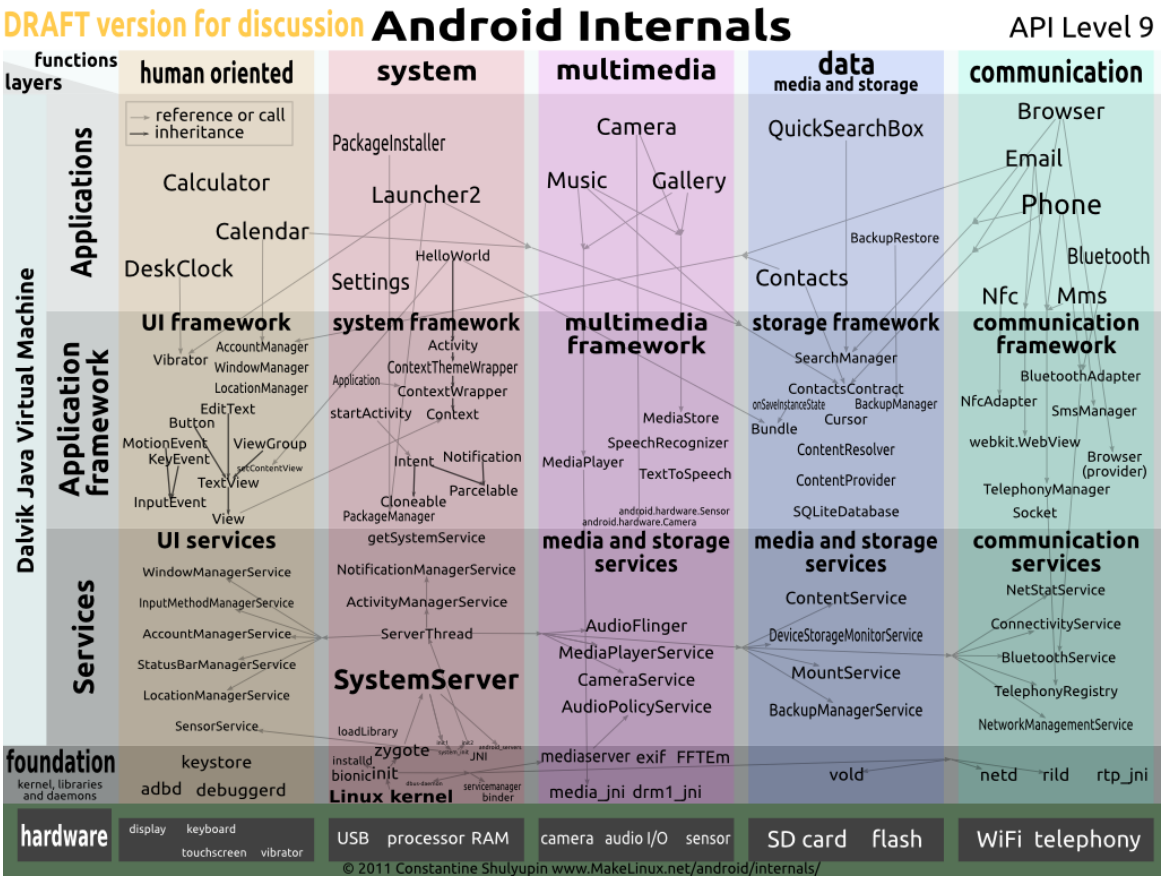


Figura 3: Detalhes da arquitetura interna do sistema *Android*. *Fonte: http://developer.android.com/.*

- **Logger:** O *Android* possui um sistema global de *logs*, implementado através de um módulo do *kernel*. É criado quatro arquivos de dispositivo em "/dev/log", onde cada um representa um *buffer* diferente.

  Para as aplicações acessarem o sistema de *log*, deve-se abrir, escrever ou ler num destes arquivos de dispositivo. A implementação deste módulo no *kernel* encontra-se em "drivers/misc/logger.c". Um exemplo pode ser visto na Figura 4.

- **Wakelocks:** Se um dispositivo *Android* ficar um tempo sem ser usado, entrará em modo de baixo consumo para garantir economia de bateria, visto que a energia em dispositivos móveis é um recurso escasso, diferente de um computador pessoal que é o principal alvo do sistema *Linux*. O módulo de *wakelock* permite que as aplicações desabilitem o mecanismo de baixo consumo. Por exemplo, se você precisar executar um processo em plano de fundo que não pode ser interrompido para entrar em modo de baixo consumo, este módulo possibilita a desativação temporária deste recurso até que seu processo finalize a execução. Sua implementação

encontra-se em “kernel/power/wakelock.c”.

- **Oom handling:** Faz o controle do uso de memória do sistema operacional e encerra processos se verificar que a memória disponível esta abaixo de um valor mínimo aceitável. É implementado em

  ‘‘drivers/misc/lowmemorykiller.c’’.

- **Timed GPIO**: É o que possibilita acionar saídas de *Input/Output (I/O)*–Entrada e Saída de forma temporizada. Está implementado em

  ‘‘drives/misc/timed_gpio.c’’.

```
# ls -l /dev/log
crw-rw--w- root     log       10,  54 1970-01-01 00:00 system
crw-rw--w- root     log       10,  55 1970-01-01 00:00 radio
crw-rw--w- root     log       10,  56 1970-01-01 00:00 events
crw-rw--w- root     log       10,  57 1970-01-01 00:00 main
```

Figura 4: Exemplo de log no *Android*. *Fonte: [11].*

3.2 O sistema de arquivos

Uma árvore de diretórios de um sistema de arquivos para o *Android* pode ser visto na Figura 5.

Novamente podemos notar um grande diferença se comparado ao sistema *Linux* usual. Os dois principais diretórios são o “data”, que armazena os dados das aplicações, e o “system”, com as bibliotecas (system/lib), serviços (system/bin e system/xbin) e aplicações *Java* (system/app).

O *Android* implementou uma biblioteca chamada *Bionic* para usar como biblioteca do sistema. O *Google* tem problemas burocráticos com licenças *GPL*. A *Bionic* usa a licença *BSD*, e suporta as arquiteturas *x86* e *ARM*. Diferentemente do *Linux*, as bibliotecas não são a *glibc* ou *uClibc*.

Uma alternativa viável seria utilizar a licença *GPL* da *Busybox*, no entanto os engenheiros da *Google* preferiram usar a *Toolbox*, uma implementação no mesmo esquema do *Busybox*, que também traz um conjunto mais limitado de comandos e ferramentas úteis para gerenciar um sistema *Android*. Podemos ver os executáveis que fazem um *link* para a Toolbox na listagem na Figura 6.

O *Android* ainda usa por padrão o *SQLite* como gerenciador de banco de dados para as aplicações e o *OpenGL/ES* como biblioteca para interface gráfica, dentre outras bibliotecas disponíveis.

```
$ sudo tree -d -L 2 .
.
|-- acct
|   `-- uid
|-- cache
|   `-- lost+found
|-- config
|-- d -> /sys/kernel/debug
|-- data
|   |-- anr
|   |-- app
|   |-- app-private
|   |-- backup
|   |-- bootchart
|   |-- dalvik-cache
|   |-- data
|   |-- dontpanic
|   |-- local
|   |-- lost+found
|   |-- misc
|   |-- property
|   |-- secure
|   |-- system
|   `-- tombstones
|-- dev
|-- mnt
|   |-- asec
|   |-- sdcard
|   `-- secure
|-- part-3
|-- proc
|-- sbin
|-- sys
`-- system
    |-- app
    |-- bin
    |-- etc
    |-- fonts
    |-- framework
    |-- lib
    |-- media
    |-- ti-dsp
    |-- usr
    `-- xbin
```

Figura 5: Exemplo de uma árvore de diretórios de um sistema de arquivos no *Android*. *Fonte: [11].*

3.3 Os serviços básicos

Se listarmos os processos rodando em um dispositivo com *Android*, a saída será mais ou menos como o visto na Figura 7.

```
$ ls -l /system/bin
lrwxrwxrwx system   system            2011-08-13 04:24 reboot -> toolbox
-rwxr-xr-x system   system       9648 2011-08-13 04:24 audioloop
-rwxr-xr-x system   system     100916 2011-08-13 04:24 iptables
-rwxr-xr-x system   system       9748 2011-08-13 04:24 sdcard
lrwxrwxrwx system   system            2011-08-13 04:24 ps -> toolbox
lrwxrwxrwx system   system            2011-08-13 04:24 top -> toolbox
-rw-r--r-- system   system       5372 2011-08-13 04:24 testwrap
lrwxrwxrwx system   system            2011-08-13 04:24 mount -> toolbox
-rwxr-xr-x system   system       5548 2011-08-13 04:24 dvz
-rwxr-xr-x system   system      18112 2011-08-13 04:24 debuggerd
lrwxrwxrwx system   system            2011-08-13 04:24 ln -> toolbox
lrwxrwxrwx system   system            2011-08-13 04:24 ifconfig -> toolbox
lrwxrwxrwx system   system            2011-08-13 04:24 renice -> toolbox
-rwxr-xr-x system   system      10792 2011-08-13 04:24 showlease
drwxr-xr-x system   system            2011-08-13 04:24 sgx
lrwxrwxrwx system   system            2011-08-13 04:24 start -> toolbox
...
```

Figura 6: Executáveis que fazem um link para o *Toolbox*. *Fonte: [11].*

Conclui-se que:

1. "O processo "init" é o pai de todos os processos que rodam em modo usuário, como o "/sbin/ueventd" e o "/system/bin/mediaserver";

2. O processo "kthreadd" é o pai de todas as *threads* do *kernel* como o "ksoftirqd" e o "khelper";

3. O processo "zygote" é o pai de todas as aplicações rodando no *Android*, como o "android.process.media" e o "com.android.email". Tudo que roda em cima de uma máquina virtual, tem como antecessor comum o processo "zygote" [11]".

3.4 As aplicações

As aplicações são escritas em linguagem *Java*. Há bibliotecas *Java* disponíveis para acessar todos os recursos do dispositivo.

Ao executar uma aplicação, o processo "zygote" cria uma instância da MV *Dalvik* para executá-la. E cada aplicação roda com um *user ID* diferente. Como vemos na coluna "USER" da listagem de processos da Figura 7. Isso protege os processos uns dos outros, e do sistema como um todo. Isso permite limitar o acesso aos arquivos do sistema e aos recursos do dispositivo através de permissões de usuário.

```
# ps
USER     PID   PPID  VSIZE  RSS     WCHAN    PC         NAME
root      1     0     504    416   c00c1b9c 0000877c S /init
root      2     0     0      0     c0072c34 00000000 S kthreadd
root      3     2     0      0     c00656f0 00000000 S ksoftirqd/0
root      4     2     0      0     c008a23c 00000000 S watchdog/0
root      5     2     0      0     c006fc14 00000000 S events/0
root      6     2     0      0     c006fc14 00000000 S khelper
root      10    2     0      0     c0078bc8 00000000 S async/mgr
root      13    2     0      0     c006fc14 00000000 S suspend
root      229   2     0      0     c009ef8c 00000000 S sync_supers
root      231   2     0      0     c009f7bc 00000000 S bdi-default
root      233   2     0      0     c006fc14 00000000 S kblockd/0
root      246   2     0      0     c006fc14 00000000 S ksuspend_usbd
root      250   2     0      0     c0240bc4 00000000 S khubd
root      253   2     0      0     c0269074 00000000 S kseriod
root      261   2     0      0     c006fc14 00000000 S twl4030-irqchip
root      262   2     0      0     c0207a40 00000000 S twl4030-irq
root      292   2     0      0     c006fc14 00000000 S kmmcd
root      308   2     0      0     c006fc14 00000000 S musb_hdrc
root      310   2     0      0     c006fc14 00000000 S rpciod/0
root      319   2     0      0     c008a434 00000000 S khungtaskd
root      320   2     0      0     c009a628 00000000 S kswapd0
root      322   2     0      0     c006fc14 00000000 S aio/0
root      323   2     0      0     c006fc14 00000000 S nfsiod
root      324   2     0      0     c006fc14 00000000 S crypto/0
root      474   2     0      0     c021a7f0 00000000 S mtdblockd
root      546   2     0      0     c006fc14 00000000 S kondemand/0
root      547   2     0      0     c006fc14 00000000 S kconservative/0
```

Figura 7: Exemplo de processos rodando no *Android*. *Fonte: [11].*

3.5 O processo de boot

O *kernel* do *Linux* executa o processo "init", que faz as configurações básicas do sistema operacional *Android* e inicia outros processos e serviços, incluindo o *zygote*, que é responsável por inicializar a MV *Dalvik* e todos os processos e serviços *Java*.

Normalmente a *CPU* possui um código de *boot* residente em *ROM* (*bootloader* de primeiro nível), responsável por carregar um *bootloader* de segundo nível para a memória. Este *bootloader* pode ainda carregar um outro *bootloader* de terceiro nível antes de iniciar o *kernel* do *Linux*. Na *beagleboard*, podemos ver o que acontece na Figura 8.

1. "O *bootloader* de primeiro estágio, residente em *ROM*, procura por imagens de *boot* em diversos dispositivos de armazenamento e carrega para a *SRAM* (limite de 64KB). Neste caso encontrará o *X–Loader* na *flash NAND*;

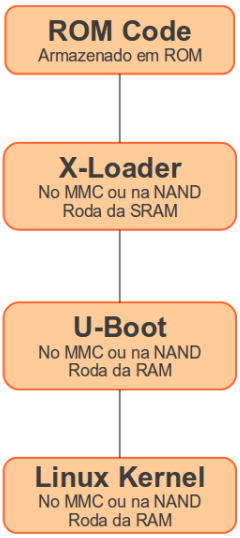


Figura 8: Esquema da *beagleboard*. *Fonte: [11].*

2. O *X–Loader*, *bootloader* de segundo estágio, é carregado para a *SRAM*. Ele inicializa a controladora da *DRAM*, *flash NAND* e leitora *MMC*, e carrega o *bootloader* de terceiro estágio, que é o *U–Boot*;

3. O *U–Boot*, *bootloader* de terceiro estágio, roda da *RAM*. Ele inicializa alguns dispositivos de hardware (rede, *USB*, etc.), carrega a imagem do *kernel* do *Linux* na *RAM* e passa o controle para ele;

4. O *kernel* roda da *RAM* e assume o controle do sistema. Tirando o controle dos *bootloaders* [11]."

Sob a visão do sistema operacional, o importante é saber que um *bootloader* será carregado em algum momento de sua execução, irá inicializar o *hardware*, executar rotinas de testes, carregar o *kernel* para a memória e executá-lo.

O processo de *boot* do *kernel* do *Linux* é muito semelhante, independentemente da plataforma ser *Android*, *Meego*, *Ubuntu* ou outra distribuição qualquer. O importante é saber que o *kernel* fará seu trabalho normalmente, inicializando processos, memória virtual, dispositivos, *drivers*, sistemas de arquivos, etc.

3.5.1 O init

Como já mencionado anteriomente, o *Android* implementa seu próprio mecanismo de inicialização dos processos básicos do sistema.

A implementação do "init" encontra–se nos códigos fontes do *Android* disponível em "system/core/init/init.c". Este processo faz algumas inicializações de sistema operacional, como por exemplo: criar diretórios e pontos de montagem básicos do *Linux* como o /proc, o /sys e o /dev; inicializar o sistema de *log* e abrir o console.

"Uma tarefa importante deste processo é o tratamento de um arquivo de configuração chamado "init.rc". Este arquivo tem objetivos parecidos com o /etc/inittab do mecanismo de inicialização *System V*.

É no "init.rc" que esta configurada boa parte do restante da inicialização do sistema, incluindo a execução dos serviços básicos do *Android*, dentre eles:

- **Console:** Inicia o *shell ash*;
- **ServiceManager:** Inicia o *binder* (responsável pela comunicação entre os processos);
- **Vold:** *Volume daemon* — controla a montagem de volumes de mídia no sistema de arquivos;
- **Adbd:** *Android debugger bridge daemon* — servidor para comunicação com o cliente *adb*;
- **Media:** Inicia os servidores multimídia (áudio, vídeo, etc);
- **Bootsound:** Executa um arquivo de áudio no *boot*, lendo um arquivo em "/system/media/audio/ui/boot.mp3";
- **Installd:** Servidor de instalação de pacotes/aplicações" **.apk*.

Depois de interpretar este arquivo, o "init" entra em um *loop* infinito monitorando a ocorrência de eventos e a execução de processos.

Pode-se, por exemplo, configurar o "init.rc" para que um processo seja iniciado quando conectado um *pendrive* na porta *USB*, (o "init" monitora a criação do *device node* em "/dev" quando um dispositivo *USB* for conectado). [11]"

3.5.2 O Zygote

O "zygote" é o pai dos processos *Java*. Todos os aplicativos desenvolvidos em *Java* é criado por este processo, que instancia uma MV *Dalvik* para executar um processo ou serviço *Java*.

O "zygote" tem basicamente dois objetivos principais:

1. "Prover uma infraestrutura para a execução de aplicações *Java*. Primeiramente, inicia a máquina virtual *Dalvik*. Em seguida, executa um servidor que abre um *socket* que aguarda requisições para execução de aplicações *Java*. Qualquer requisição de execução de aplicações *Java* passa por esse servidor, que faz um *fork* para executar a aplicação em uma outra instância da máquina virtual. O código–fonte deste servidor esta disponível em "frameworks/base/core/java/com/android/internal/os/ZygoteConnection.java";

2. Iniciar o System Server, que gerencia a base dos serviços do sistema operacional *Android* [11]".

3.5.3 System server

A implementação do *system server* encontra-se em "frameworks/base/services/java/com/android/server/SystemServer.java". Ele inicia todos os serviços *Java* básicos do *Android*. Dentre eles temos o *Power Manager, Activity Manager, Telephony Registry, Package Manager, Context Manager, System Context Providers, Battery Service, Alarm Manager, Sensor Service, Window Manager, Bluetooth Service, Mount Service, Status Bar Service ,Hardware Service, NetStat Service, Connectivity Service, Notification Manager, DeviceStorageMonitor Service, Search Service, Clipboard Service, etc.*

Todos os serviços tem sua importância dentro do ambiente e das aplicações do sistema *Android*. O mais importante é entender que cada funcionalidade esta abstraída através de um serviço, e a *API* do *Android* disponibiliza funções para as aplicações se comunicarem com estes serviços.

Quase no final do *boot*, o *Activity Manager* inicia alguns processos básicos, dentre eles o com.android.launcher, que é a aplicação responsável pela interface gráfica no *Android*.

4 Gerenciamento de processos e memória

Como já foi dito anteriormente o *Android* é um sistema operacional baseado em *Linux* com o *kernel* 2.6.x e utiliza o gerenciamentode memória baseado em tal. Todas as operações básicas do

sistema operacional em níveis mais baixos, como o *I/O*, gerenciamento de memória, e assim por diante, são tratados pelo *kernel* do *Linux*. Sendo assim o sistema se utiliza da biblioteca padrão do C, que acompanha o *Linux* há anos.

4.1 *Android* runtime

Existem algumas peculiaridades no gerenciamento de memória do *Android*. O *Android* inclui um grupo de bibliotecas que fornece a maioria das funcionalidades disponíveis nas principais bibliotecas da linguagem *Java*. Toda aplicação *Android* possui um processo e instância próprios na máquina virtual *Dalvik*. Assim como nas plataformas *Java* e *.Net*, o *Android* utiliza-se de uma Máquina Virtual(VM) pória, para assegurar que a execução de várias instâncias em um único dispositivo seja eficiente. A *Dalvik* utiliza o *kernel* do *Linux* para lidar com funcionalidades de nível mais baixo, como segurança, *threading* e gerenciamento de processos e memória.

4.2 Dalvik Virtual Machine

Todo hardware do dispositivo e serviços do sistema operacional são controlados usando o *Dalvik* como uma camada intermediária. Através do uso desta máquina virtual para hospedar a execução de aplicativos os desenvolvedores podem abstrair da implementação em *hardwares* em particular, facilitando a criação de novos aplicativos, que podem executar em qualquer dispositivo igualmente.

A *Dalvik* possui também uma extensão própria para os arquivos executáveis, a .dex, que garante um menor consumo de memória. Os arquivos .dex são criados através da transformação das classes compiladas do *Java* pelas ferramentas fornecidas na *Android SDK*.

Além disso, desde a versão 2.2 (*Froyo*), o *Android* possui uma implementação de *Just–in–time (JIT)*, que compila *dexcodes* para a arquitetura–alvo em tempo de execução, tornando a execução dos processos consideravelmente mais rápidas, já que não precisa ficar interpretando *dexcodes*.

Diferentemente da *Java VM* e *.Net*, o *Android* também gerencia o tempo de vida do processo. Para otimizar o uso de memória verifica-se o estado de resposta do aplicativo, parando e matando processos conforme necessário para liberar recursos à aplicações de maior prioridade.

Junto com a máquina virtual *Dalvik*, o *Android* usa o *framework Apache Harmony*, desenvolvido pela *Apache Software Fundation* como biblioteca padrão de classes *Java*.

4.2.1 Prioridade e *status* de processos

No *Android* todos os processos são mantidos na memória até que haja a necessidade de recursos para outros processos. A ordem na qual os processos são finalizados para liberação de recursos está associada ao nível de prioridade da aplicação do mesmo. A prioridade de uma aplicação é igual a de seu componente de maior prioridade. Na Figura 9 temos os possíveis estados e os níveis de prioridade com o qual os processos são tratados.

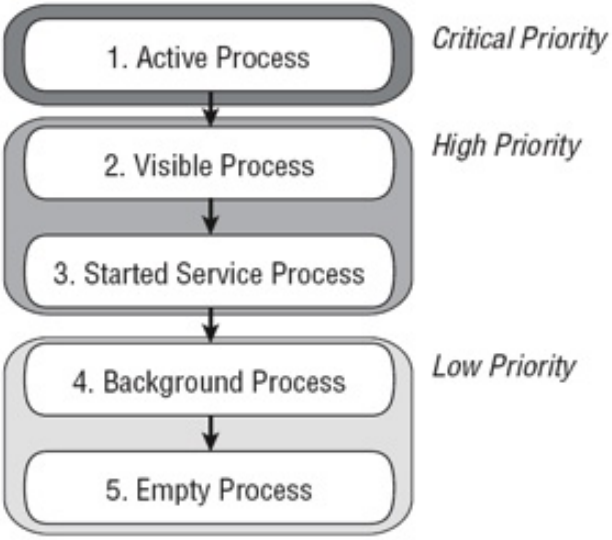


Figura 9: Estados de processos e seu nível de prioridade associado. *Fonte: [4].*

- **Active Processes**: São processos em interação com usuário (*foreground*). São os processos mantidos a todo custo pelo gerenciador;
- **Visible Processes**: São processos que estão visíveis, mas não estão interagindo com o usuário. Estes processos somente são terminados em casos de extrema necessidade, para que um processo ativo continue executando;
- **Started Service Processes**: São processos hospedando serviços que foram iniciados e não possuem interface visível. Como não interagem diretamente com o usuário possuem um nível de prioridade inferior ao de processos visíveis;
- **Background Processes**: São processos de atividades que não estão visíveis e não possuem serviços iniciados. Estes processos são finalizados utilizando-se o padrão *Last–Seen–First–Killed*;

- **Empty Processes**: São processos já finalizados, que são mantidos na memória pelo sistema operacional, para acelerar o *start–up* do mesmo e melhorar a performance do sistema. Estes processos são frequentemente finalizados para serem finalizados e cederem recursos aos processos em execução.

O nível de prioridade de um processo também é afetado pelas dependências interprocessos. Se um processo depende de um serviço ou de um conteúdo a ser provido por um outro processo, este segundo processo terá um nível de prioridade igual ou maior que o primeiro.

Quando dois processos possuem mesmo nível de prioridade, como critério de desempate o processo que teve histórico de prioridade mais baixa durante mais tempo é escolhido para ser finalizado primeiro.

4.3 Memória Virtual

"Memória virtual, é uma técnica que usa a memória principal como uma *cache* para armazenamento secundário. Houve duas motivações principais: permitir o compartilhamento seguro e eficiente da memória entre vários programas e remover os transtornos de programação de uma quantidade pequena e limitada na memória principal. A memória virtual consiste em recursos de hardware e software com três funções básicas:

- **Realocação:** Que assegura que cada processo (aplicação) tenha o seu próprio espaço de endereçamento, começando em zero;
- **Proteção:** Impede que um processo utilize um endereço de memória que não lhe pertença;
- **Paginação (paging) ou troca (swapping):** Que possibilita a uma aplicação utilizar mais memória do que a fisicamente existente.

Simplificadamente, um usuário ou programador vê um espaço de endereçamento virtual, que pode ser igual, maior ou menor que a memória física [9]".

Como foi dito anteriormente, o gerenciamento de memória em baixo nível do *Android* é feito pelo *Linux Kernel* 2.6. A descrição da memória virtual é feita portanto no *Linux* através de segmentação e paginação. Faremos uma breve descrição de ambos a seguir.

4.3.1 Segmentação

A segmentação divide a memória em 2 espaços distintos, o espaço do *kernel* (*Kernel Space*) e o espaço do usuário (*User Space*). Dentro destes espaços temos os 4 segmentos básicos:

- *Kernel Code.*
- *Kernel Data/Stack.*
- *User Code.*
- *User Data/Stack.*

Desta forma podemos garantir a proteção da memória, evitando o acesso de memória entre usuários. Garantindo principalmente que processos em modo *kernel* não se misturem com processos em modo usuário e que a pilha de dados não cresça indiscriminadamente.

4.3.2 Paginação

Somente com a segmentação teríamos blocos de memória contínuos para cada processo. Isso sobrecarregaria a memória, copiando a imagem de um processo todo de uma vez. Caso não haja um espaço de memória suficiente para alocar todo o processo, ou seja, não há um segmento que o comportasse e haverá falha por falta de segmento (*segmentation fault*). Para resolver tal problema é usada a paginação, onde a memória é dividida em pedaços de tamanho fixo (páginas), e segmentos de código são alocados nestes e mapeados utilizando-se uma tabela de páginas, ao invés de alocação de todo código de uma única vez.

No *Linux* a paginação é feita em 3 níveis, ou seja, são usadas 3 tabelas para mapear a memória, a *Page Directory*, *Page Middle Directory* e *Page Table*. O campo *directory field* é usado como índice para o diretório global, que existe para cada processo. O valor achado nessa posição é um ponteiro para a *page middle table*, que é novamente indexada e contém um ponteiro que indica para o endereço virtual de memória. Isso pode ser visto na Figura 10. Além disso, para aumentar o desempenho do sistema é mantido um *buffer* com os últimos endereços acessados, para que não haja necessidade de fazer múltiplos acessos as páginas. Este *buffer* é chamado de *TLB* (*Translation Lookaside Buffer*). Logo antes de realizar a procura do endereço utilizando as tabelas de página o sistema busca a tradução direta do endereço no *TLB*. É importante que a *TLB* se mantenha atualizada com relação

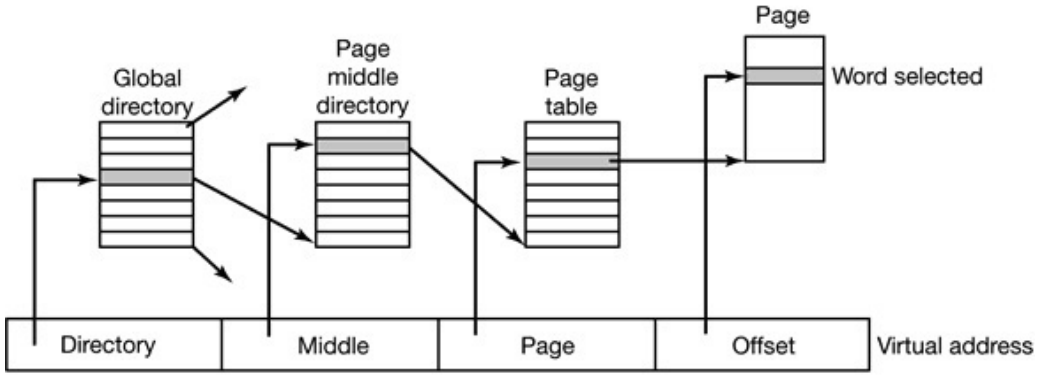


Figura 10: Paginação no *Linux*.

ao conteúdo da memória principal. Como a TLB, em geral, possui tamanho menor que a memória principal, é necessário substituir alguma tradução armazenada por outra mais recente.

4.3.3 Substituição de páginas

Quando a memória já preencheu todas as páginas possíveis é necessário realizar a substituição de páginas quando o processo em execução requisita que uma nova página seja alocada. Para tal existem diversos algoritmos de substituição paara escolher qual página deverá ser removida para a entrada da nova página na memória. Na Figura 11 vemos a seguência de ações desenvolvidas por um sistema operacional quando da solicitação de uma página, que envolve os conceitos de tabela de páginas e *TLB*.

Alguns algoritmos foram criados para otimizar esta substituição. O algoritmo utilizado no *Linux* é o *Least Recentment Used* (LRU).

O LRU remove da memória a página que não foi utilizada por mais tempo. Isso baseia-se na suposição de que páginas que não foram recentemente utilizadas também não o serão nas próximas instruções, enquanto páginas bastante utilizadas agora tendem a ser bastante utilizadas na instruções seguintes.

4.4 Comunicação entre processos

Como a memória é segmentada e paginada, o acesso de um processo à memória que não lhe pertence não é permitida. Por isso foram criados mecanismos de comunicação entre processos. Para

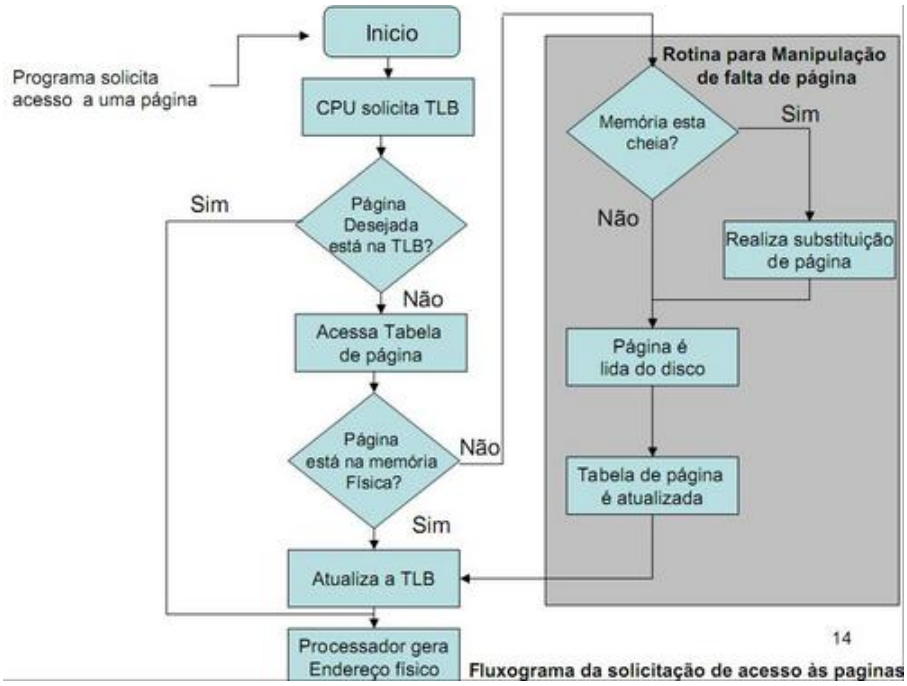


Figura 11: Fluxograma de solicitação de página.

os sistemas *Linux* tradicionais usa-se o padrão *System V IPC* para comunicação entre processos (*message queues*, semáforos e *shared memory*). Já no *Android* usa-se o *binder* para a comunicação entre processos. Ele implementa um módulo no *kernel* em "drivers/misc/binder.c" para esta tarefa. Toda comunicação entre processos no *Android* passa pelo binder. Para o desenvolvedor de aplicações *Android*, o processo é transparente, já que é abstraído pelas bibliotecas do sistema.

4.5 Escalonamento de *CPU*

O problema básico de escalonamento em sistemas operacionais é como satisfazer simultaneamente objetivos conflitantes: tempo de resposta rápido, bom *throughput* para processos em segundo plano, evitar postergação indefinida, conciliar processos de alta prioridade com de baixa prioridade, etc. O conjunto de regras utilizado para determinar como, quando e qual processo deverá ser executado é conhecido como política de escalonamento. Assim como no *Linux*, o sistema operacional *Android* divide os seus processos em três grandes classe: processos interativos, processos *batch* e processos tempo real. Em cada classe, os processos podem ser ainda subdivididos em *I/O bound* ou *CPU bound* de acordo com a proporção de tempo que ficam esperando por operações de entrada e saída ou utilizando o processador. O escalonador do *Android* não distingue processos interativos de processos *batch*, diferenciando-os apenas dos processos em tempo real. O escalonador do sistema

Android é baseado em *time-sharing*, ou seja, o tempo do processador é dividido em fatias de tempo, fatias essas denominadas de *quantum*, as quais são alocadas aos processos. Se, durante a execução de um processo, o *quantum* é esgotado, um novo processo é selecionado para execução, provocando então uma troca de contexto. Esse procedimento é completamente transparente ao processo e baseia-se em interrupções de tempo. Esse comportamento confere ao *Android* um escalonamento do tipo preemptivo. O algoritmo de escalonamento do *Android* divide o tempo de processamento em épocas (*epochs*). Cada processo, no momento de sua criação, recebe um *quantum* calculado no início de uma época. Diferentes processos podem possuir diferentes valores de *quantum*. Outra característica do escalonador do sistema *Android* é a existência de prioridades dinâmicas. O escalonador do monitora o comportamento de um processo e ajusta dinamicamente sua prioridade, visando a equalizar o uso do processador entre os processos. Processos que recentemente ocuparam o processador durante um período de tempo considerado "longo" têm sua prioridade reduzida. De forma análoga, aqueles que estão há muito tempo sem executar recebem um aumento na sua prioridade, sendo então beneficiados em novas operações de escalonamento [17].

4.6 *Deadlocks*

Um *deadlock* é caracterizado por uma situação em que ocorre um impasse entre dois ou mais processos que ficam impedidos de continuar suas execuções, ou seja, ficam bloqueados. O *deadlock* ocorre com um conjunto de processos e recursos não-preemptíveis, onde um ou mais processos desse conjunto está aguardando a liberação de um recurso por um outro processo que, por sua vez, aguarda a liberação de outro recurso alocado ou dependente do primeiro processo [15].

Assim como no *Linux*, no *Android* não é feito tratamento de *deadlocks*. O sistema parte do princípio de que eles não virão a ocorrer.

5 Desenvolvimento no *Android*

Existem diversas ferramentas para auxiliar no desenvolvimento de aplicativos para o sistema *Android*. A mais completa é o *kit* de desenvolvimento *Android SDK*, que contém um ambiente com as características e especificações do *Android*. Dentre as vantagens no desenvolvimento *Android*, estão as *APIs* (*Application Programming Interface*) com diversas funcionalidades provendo os serviços do

sistema.

As *APIs* estabelecidas para o *Android* permitem total modificação por meio de programação do seu conteúdo. Porém, programas que não precisa envolver-se em detalhes da implementação do *software* podem apenas utilizar os serviços, se a preocupação de como funciona, utilizando apenas as características menos evidentes ao usuário padrão. Um ponto forte das *APIs* básicas do *Android* é a otimização que estas possuem, focando a utilidade dos pacotes, em conjunto com um bom aproveitamento, deixando de fora pacotes pesados e pouco evoluídos. Através destas, pode ser criada toda a interface com o usuário, permitindo a criação de telas, acessar arquivos, criptografar dados, ou seja, utilizar a funcionalidade definida pelo utilizador.

Entre as principais *APIs* podemos destacar:

- **Location Manager (android.maps):** Usada para obter a posição geográfica do usuário. Como por exemplo, em aplicações que fazem uso de *GPS*;
- **Telephony Manager (android.telephony):** Informações sobre dispositivos como bateria e serviços de telefonia celular podem ser obtidos através dessa *API*;
- **Window Manager (android.view):** Responsável pelo gerenciamento de toda janela de uma aplicação, principais funções e componentes de inteface gráfica;
- **Content Providers (android.provider):** Responsável pela disponibilização dos dados através das aplicações tornando esses dados públicos. Quase todo tipo de dado é compartilhável, como áudio, vídeo, imagens e texto;
- **Resource Manager (android.util):** Todos os recursos que uma aplicação irá usar como áudio, vídeo, arquivos *XML*, são separados dela a fim de que sejam otimizados para ocupar menos espaço e demorar menos tempo para que sejam carregados. Essa *API* facilita o acesso a esses recursos;
- **Notification Manager:** Permite que uma aplicação exiba notificações, ative *LEDs*, luzes, sons ou vibração disponíveis no dispositivo;
- **Activity Manager:** Responsável pelo gerenciamento de cada atividade do sistema. No *Android* cada atividade é gerenciada através de uma pilha de atividades. Toda nova atividade criada

vai para o topo de pilha de atividades e se torna uma *running activity*, significando que será executada;

- **Webkit (android.webkit):** Inclui *APIs* para conteúdo *web*, bem como um navegador embutido para utilização geral;
- **android.app:** *APIs* de Alto nível referentes ao modelo da aplicação;
- **android.widget:** Contém *widgets* prontos (botões, listas, grades, etc) para serem utilizados nas aplicações;
- **android.database:** Contém as APIs para comunicação com o banco de dados SQLite;
- **android.os:** Contém serviços referentes ao sistema operacional, passagem de parâmetros e comunicação entre processos.

5.1 Android.os

A *API Android.os* provê os serviços básicos e essenciais do sistema operacional, além de serviços de troca de mensagens e a comunicação entre processos. As funções da *API* podem ser observadas no Apêndice A.

6 Considerações finais

No decorrer desse trabalho foram destacadas as principais características do sistema operacional *Android* dando ênfase ao bom momento em que se encontra o mercado atual dos dispositivos móveis. Com um evidente crescimento no desenvolvimento de dispositivos móveis, o desenvolvimento de aplicativos para *smartphones* se torna um grande nicho de mercado que vem crescendo exponencialmente. O sistema operacional *Android* se tornou um atrativo para esse mercado ávido por tecnologia, principalmente pelo seu conceito de integração e flexibilidade embutidos, fornecendo total acesso aos programadores que desenvolvem aplicativos para essa plataforma.

A Tabela de funções da *API* android.os

Handler.Callback	Callback interface you can use when instantiating a Handler to avoid having to implement your own subclass of Handler.
IBinder	Base interface for a remotable object, the core part of a lightweight remote procedure call mechanism designed for high performance when performing in-process and cross-process calls.
IBinder.DeathRecipient	Interface for receiving a callback when the process hosting an IBinder has gone away.
IInterface	Base class for Binder interfaces.
MessageQueue.IdleHandler	Callback interface for discovering when a thread is going to block waiting for more messages.
Parcelable	Interface for classes whose instances can be written to and restored from a `Parcel`.
Parcelable.ClassLoaderCreator<T>	Specialization of `Parcelable.Creator` that allows you to receive the ClassLoader the object is being created in.
Parcelable.Creator<T>	Interface that must be implemented and provided as a public CREATOR field that generates instances of your Parcelable class from a Parcel.
RecoverySystem.ProgressListener	Interface definition for a callback to be invoked regularly as verification proceeds.

Figura 12: Funções de interfaces da *API* android.os. *Fonte: http://developer.android.com/.*

BadParcelableException	The object you are calling has died, because its hosting process no longer exists.
DeadObjectException	The object you are calling has died, because its hosting process no longer exists.
NetworkOnMainThreadException	The exception that is thrown when an application attempts to perform a networking operation on its main thread.
ParcelFormatException	The contents of a Parcel (usually during unmarshalling) does not contain the expected data.
RemoteException	Parent exception for all Binder remote-invocation errors
TransactionTooLargeException	The Binder transaction failed because it was too large.

Figura 13: Funções de exceptions da *API* android.os. *Fonte: http://developer.android.com/.*

AsyncTask.Status	Indicates the current status of the task.

Figura 14: Funções de enums da *API* android.os. *Fonte: http://developer.android.com/.*

AsyncTask<Params, Progress, Result>	AsyncTask enables proper and easy use of the UI thread.
BatteryManager	The BatteryManager class contains strings and constants used for values in the `ACTION_BATTERY_CHANGED` Intent.
Binder	Base class for a remotable object, the core part of a lightweight remote procedure call mechanism defined by `IBinder`.
Build	Information about the current build, extracted from system properties.
Build.VERSION	Various version strings.
Build.VERSION_CODES	Enumeration of the currently known SDK version codes.
Bundle	A mapping from String values to various Parcelable types.
ConditionVariable	Class that implements the condition variable locking paradigm.
CountDownTimer	Schedule a countdown until a time in the future, with regular notifications on intervals along the way.
Debug	Provides various debugging functions for Android applications, including tracing and allocation counts.
Debug.InstructionCount	API for gathering and querying instruction counts.
Debug.MemoryInfo	This class is used to retrieved various statistics about the memory mappings for this process.
DropBoxManager	Enqueues chunks of data (from various sources -- application crashes, kernel log records, etc.).
DropBoxManager.Entry	A single entry retrieved from the drop box.
Environment	Provides access to environment variables.
FileObserver	Monitors files (using inotify) to fire an event after files are accessed or changed by by any process on the device (including this one).
Handler	A Handler allows you to send and process `Message` and Runnable objects associated with a thread's `MessageQueue`.
HandlerThread	Handy class for starting a new thread that has a looper.
Looper	Class used to run a message loop for a thread.
MemoryFile	MemoryFile is a wrapper for the Linux ashmem driver.
Message	Defines a message containing a description and arbitrary data object that can be sent to a `Handler`.
MessageQueue	Low-level class holding the list of messages to be dispatched by a `Looper`.
Messenger	Reference to a Handler, which others can use to send messages to it.
Parcel	Container for a message (data and object references) that can be sent through an IBinder.
ParcelFileDescriptor	The FileDescriptor returned by `readFileDescriptor()`, allowing you to close it when done with it.
ParcelFileDescriptor.AutoCloseInputStream	An InputStream you can create on a ParcelFileDescriptor, which will take care of calling `ParcelFileDescriptor.close()` for you when the stream is closed.
ParcelFileDescriptor.AutoCloseOutputStream	An OutputStream you can create on a ParcelFileDescriptor, which will take care of calling `ParcelFileDescriptor.close()` for you when the stream is closed.
ParcelUuid	This class is a Parcelable wrapper around `UUID` which is an immutable representation of a 128-bit universally unique identifier.
PatternMatcher	A simple pattern matcher, which is safe to use on untrusted data: it does not provide full reg-exp support, only simple globbing that can not be used maliciously.
PowerManager	This class gives you control of the power state of the device.
PowerManager.WakeLock	Class lets you say that you need to have the device on.
Process	Tools for managing OS processes.
RecoverySystem	RecoverySystem contains methods for interacting with the Android recovery system (the separate partition that can be used to install system updates, wipe user data, etc.)
RemoteCallbackList<E extends IInterface>	Takes care of the grunt work of maintaining a list of remote interfaces, typically for the use of performing callbacks from a `Service` to its clients.
ResultReceiver	Generic interface for receiving a callback result from someone.
StatFs	Retrieve overall information about the space on a filesystem.
StrictMode	StrictMode is a developer tool which detects things you might be doing by accident and brings them to your attention so you can fix them.
StrictMode.ThreadPolicy	`StrictMode` policy applied to a certain thread.
StrictMode.ThreadPolicy.Builder	Creates `StrictMode.ThreadPolicy` instances.
StrictMode.VmPolicy	`StrictMode` policy applied to all threads in the virtual machine's process.
StrictMode.VmPolicy.Builder	Creates `StrictMode.VmPolicy` instances.
SystemClock	Core timekeeping facilities.
TokenWatcher	Helper class that helps you use IBinder objects as reference counted tokens.
Vibrator	Class that operates the vibrator on the device.
WorkSource	Describes the source of some work that may be done by someone else.

Figura 15: Funções de classes da *API* android.os. *Fonte: http://developer.android.com/.*

Referências

[1] Silberschatz, A.; Galvin, P.B.; Gagne, G. – **"Fundamento de Sistemas Operacionais"**. 8 ed., LTC, 2010.

[2] Android.com – **"Discover Android"**.

Disponível em: `http://www.android.com/about/`

Acesso em 17 de maio de 2012.

[3] Businessweek.com – **"Google Buys Android for Its Mobile Arsenal"**.

Disponível em:

`http://www.businessweek.com/technology/content/aug2005/tc20050817_0949_tc024.htm`

Acesso em 11 de maio de 2012.

[4] Mobworld – **"Memory Management in Android"**.

Disponível em:

`http://mobworld.wordpress.com/2010/07/05/memory-management-in-android/`

Acesso em 14 de maio de 2012.

[5] HowStuffWorks – **"Como Funciona A Memória Virtual"**.

Disponível em: `http://informatica.hsw.uol.com.br/memoria-virtual.htm`

Acesso em 18 de maio de 2012.

[6] Wikipédia – **"Android"**.

Disponível em: `http://pt.wikipedia.org/wiki/Android`

Acesso em 15 de maio de 2012.

[7] Wikipédia – **"Sistema Operacional"**.

Disponível em: `http://pt.wikipedia.org/wiki/Sistema_operativo`

Acesso em 15 de maio de 2012.

[8] Wikipédia – **“Linux (Núcleo)”**.

Disponível em: `http://pt.wikipedia.org/wiki/Linux_(núcleo)`

Acesso em 15 de maio de 2012.

[9] Wikipédia – **“Memória Virtual”**.

Disponível em: `http://pt.wikipedia.org/wiki/Memória_virtual`

Acesso em 17 de maio de 2012.

[10] Oficina da Net – **“Tudo sobre o *Google* Android OS”**.

Disponível em:
`http://www.oficinadanet.com.br/artigo/outros_sistemas/google_android_os`

Acesso em 20 de maio de 2012.

[11] Padro, Sergio, – **“Entendendo o processo de boot do Android"** e **“Introdução ao funcionamento interno do Android”**.

Disponível em: `http://sergioprado.org/entendendo-o-processo-de-boot-do-android/`

Disponível em:
`http://sergioprado.org/introducao-ao-funcionamento-interno-do-android/`

Acesso em 18 de maio de 2012.

[12] Meier, Reto. – **“Professional Android Application Development”**. Indianapolis: Wiley Publishing, 2009. Werneck, Rafael.

[13] Nakashima, Rafaela, – **“Introdução–Embarcados Básico”**.

Disponível em: `http://embarcadosbasico.wordpress.com/2011/12/14/introducao/`

Acesso em 18 de maio de 2012.

[14] Zanuz, L.; Meneghel, M.; Marcon.F; Borba, M. – **“Escalonamento de CPU no *Linux*”**.

Disponível em: `http://lzanuz.sites.uol.com.br/escalonamento.htm`

Acesso em 20 de maio de 2012.

[15] Wikipédia – **“Deadlock”**.

Disponível em: `http://pt.wikipedia.org/wiki/Deadlock`

Acesso em 21 de maio de 2012.

[16] Pereira, L. C. O.; Silva, M. L. – **“Android Para Desenvolvedores”**. 1 ed., Brasport, 2009.

[17] de Oliveira, R.; Carissimi, A.; Toscani, S. – **“Escalonamento em Linux”**

Disponível em: `http://www.inf.ufrgs.br/~asc/livro/secao94.pdf`

Acesso em 21 de maio de 2012.